

Financial Forecasting Frontier: Distributed ML

Technical Documentation and Implementation Report

Submitted by – Anshul Baghel

1. Project Overview

This project demonstrates distributed machine learning and banking analytics using Hadoop, Hive, Apache Spark, Spark ML, Spark Streaming, and local MapReduce simulation. The system simulates a real-world distributed banking analytics environment capable of scalable batch and streaming data processing.

2. Technologies Used

- Hadoop HDFS
- Apache Hive
- Apache Spark
- PySpark
- Spark ML
- Spark Structured Streaming
- Python
- Java 8
- Windows 11

3. Dataset Description

Dataset Used: bank.csv

Target Variable:

y -> Indicates whether the client subscribed to a term deposit.

Total Rows: 4521

Total Columns: 17

4. System Architecture

Raw Data → HDFS → Spark Processing → Spark ML → Spark Streaming → Hive Analytics → Business Insights

5. Hadoop and HDFS Setup

HDFS was configured locally on Windows 11. NameNode, DataNode, ResourceManager, and NodeManager services were successfully started and verified using JPS.

6. Spark Data Processing and EDA

Exploratory Data Analysis was performed using Spark DataFrames.

Tasks performed:

- Dataset loading
- Schema inspection
- Partition analysis
- GroupBy aggregations
- Feature engineering
- Spark SQL queries
- Visualizations

7. Data Parallelism

The dataset was repartitioned to improve parallel execution. Spark distributed transformations and aggregations were used to demonstrate scalable processing.

8. Machine Learning Pipeline

A distributed ML pipeline was implemented using Spark ML.

Pipeline Steps:

- Missing value handling
- StringIndexer
- OneHotEncoder
- VectorAssembler
- Logistic Regression
- AUC Evaluation
- Model Saving

9. Real-Time Streaming Analytics

Spark Structured Streaming was implemented using file-based stream simulation.

Features implemented:

- Real-time aggregation
- Window operations
- Watermarking

- Real-time predictions
- Checkpointing

10. Hadoop MapReduce Simulation

Local MapReduce simulation was implemented using Python mapper and reducer scripts.

Implemented Tasks:

- Average balance by job type
- Housing loan count by education
- Monthly contact analytics

11. Hive Analytics

HiveQL queries were executed for distributed SQL analytics.

Queries performed:

- Total customers
- Average age by job
- Average balance by job
- Subscription analysis
- Contact month trend
- Outcome impact analysis

12. Model Evaluation

The Logistic Regression model was evaluated using Area Under ROC Curve (AUC) and prediction accuracy.

The model successfully predicted customer subscription behaviour.

13. Business Impact

The system demonstrates how distributed analytics can help banks perform:

- Customer segmentation
- Marketing optimization
- Predictive analytics
- Real-time monitoring
- Scalable data processing

14. Future Improvements

- Kafka Integration
- Cloud Deployment

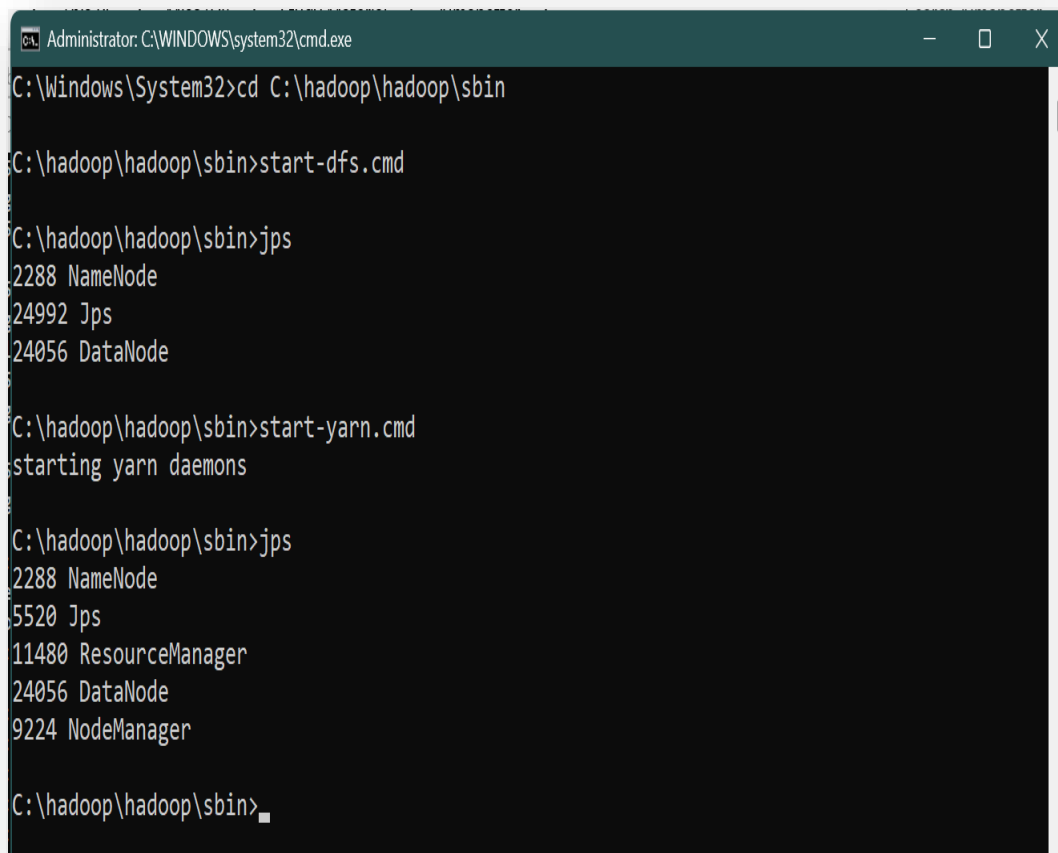
- Kubernetes
- Spark on Cloud
- MLOps Pipeline
- Real-time dashboards
- CI/CD integration

15. Conclusion

This project successfully demonstrated end-to-end distributed machine learning and analytics on banking data using modern big data technologies.

16. Screenshots Section

Screenshot 1: Hadoop Services Running (JPS Output)



```
Administrator: C:\WINDOWS\system32\cmd.exe
C:\Windows\System32>cd C:\hadoop\hadoop\sbin

C:\hadoop\hadoop\sbin>start-dfs.cmd

C:\hadoop\hadoop\sbin>jps
2288 NameNode
24992 Jps
24056 DataNode

C:\hadoop\hadoop\sbin>start-yarn.cmd
starting yarn daemons

C:\hadoop\hadoop\sbin>jps
2288 NameNode
5520 Jps
11480 ResourceManager
24056 DataNode
9224 NodeManager

C:\hadoop\hadoop\sbin>
```

Screenshot 2: HDFS Web UI

Hadoop

Overview

Datanodes

Datanode Volume Failures

Snapshot

Startup Progress

Utilities

Overview

'localhost:9820' (✔active)

Started:	Tue May 26 14:10:07 +0530 2026
Version:	3.3.6, r1be78238728da9266a4f88195058f08fd012bf9c
Compiled:	Sun Jun 18 13:52:00 +0530 2023 by ubuntu from (HEAD detached at release-3.3.6-RC1)
Cluster ID:	CID-c0088b1c-1bd0-449a-b884-912292d74cac
Block Pool ID:	BP-1413568643-192.168.29.164-1779300741701

In operation

DataNode State

All

Show

25

entries

Search:

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Version
✔/default-rack/host.docker.internal:9866 (127.0.0.1:9866)	http://host.docker.internal:9864	2s	7m	8.63 KB	238.61 GB	275.96 GB	2	8.63 KB (0%)	3.3.6

Showing 1 to 1 of 1 entries

Previous

1

Next

Screenshot 3: Spark Session Started

```
import os
import sys
import findspark

# Reset old spark session if exists
try:
    spark.stop()
except:
    pass

os.environ["JAVA_HOME"] = r"C:\Java\jdk1.8.0_491"
os.environ["SPARK_HOME"] = r"C:\spark\spark-3.5.8-bin-hadoop3"
os.environ["HADOOP_HOME"] = r"C:\hadoop"

findspark.init(
    r"C:\spark\spark-3.5.8-bin-hadoop3"
)

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .master("local[*]") \
    .appName("BankDistributedML") \
    .config(
        "spark.sql.warehouse.dir",
        "file:///C:/spark-warehouse"
    ) \
    .enableHiveSupport() \
    .getOrCreate()

print("Spark started")
```

✓ 5.6s

Spark Started

Screenshot 4: Dataset Loaded in Spark

Dataset Loading

```
df = spark.read.csv(  
    r"D:\Almabetter\ML2. Financial-Forecasting-Distributed-ML\data\raw\bank.csv",  
    header=True,  
    inferSchema=True  
)  
  
df.show(5)
```

✓ 8.0s

Python

age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
30	unemployed	married	primary	no	1787	no	no	cellular	19	oct	79	1	-1	0	unknown	no
33	services	married	secondary	no	4789	yes	yes	cellular	11	may	220	1	339	4	failure	no
35	management	single	tertiary	no	1350	yes	no	cellular	16	apr	185	1	330	1	failure	no
30	management	married	tertiary	no	1476	yes	yes	unknown	3	jun	199	4	-1	0	unknown	no
59	blue-collar	married	secondary	no	0	yes	no	unknown	5	may	226	1	-1	0	unknown	no

only showing top 5 rows

Screenshot 5: Dataset Schema

```
df.printSchema()
```

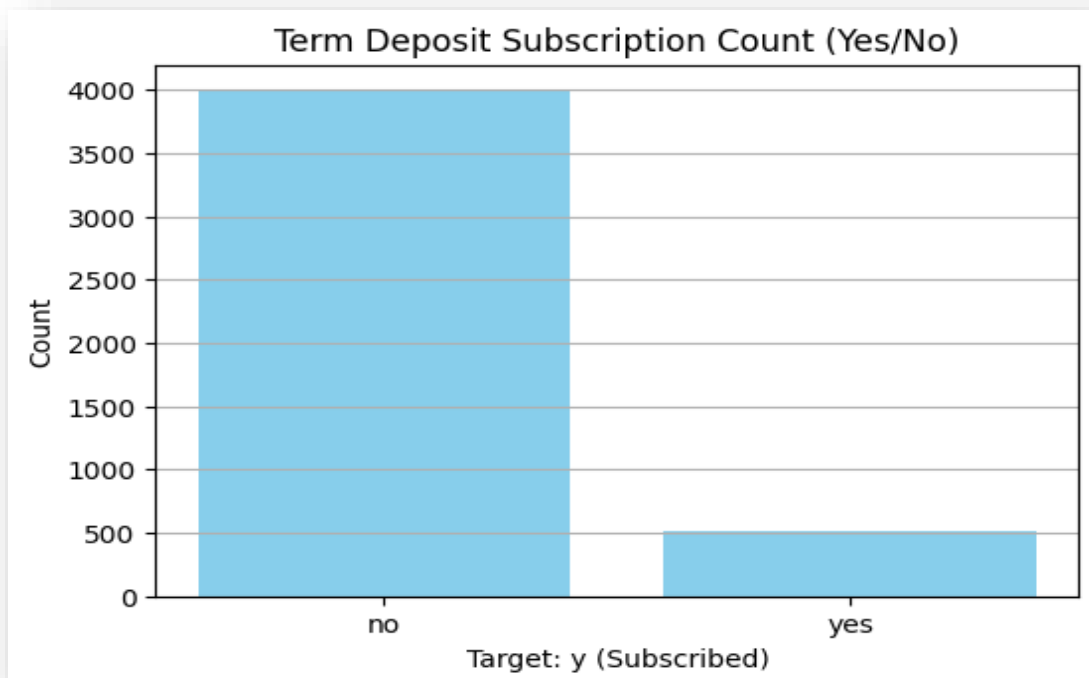
[3] ✓ 0.0s

```
... root
  |-- age: integer (nullable = true)
  |-- job: string (nullable = true)
  |-- marital: string (nullable = true)
  |-- education: string (nullable = true)
  |-- default: string (nullable = true)
  |-- balance: integer (nullable = true)
  |-- housing: string (nullable = true)
  |-- loan: string (nullable = true)
  |-- contact: string (nullable = true)
  |-- day: integer (nullable = true)
  |-- month: string (nullable = true)
  |-- duration: integer (nullable = true)
  |-- campaign: integer (nullable = true)
  |-- pdays: integer (nullable = true)
  |-- previous: integer (nullable = true)
  |-- poutcome: string (nullable = true)
  |-- y: string (nullable = true)
```

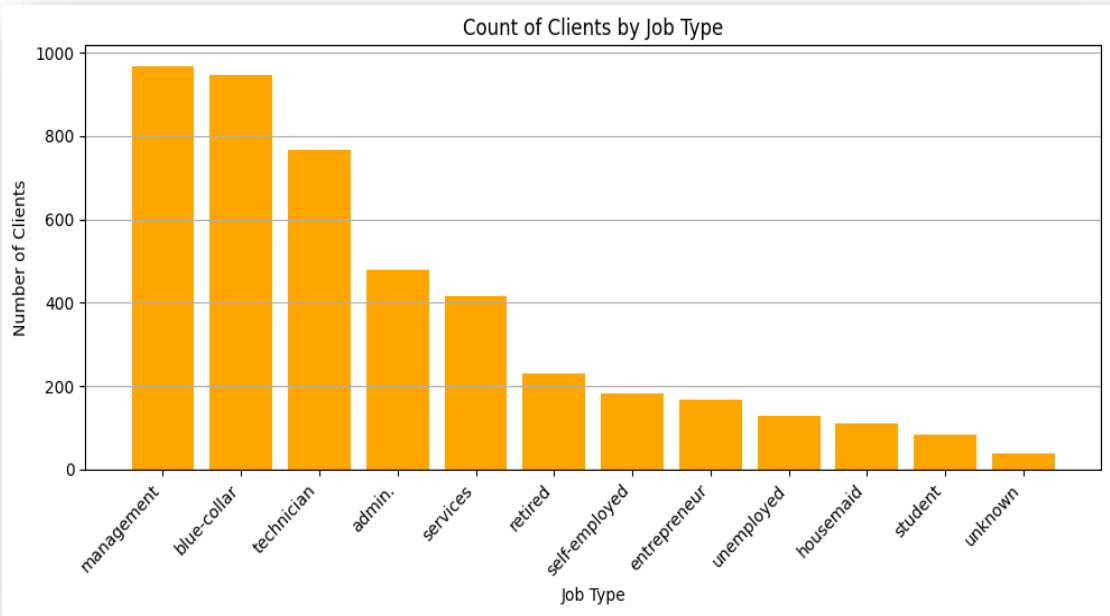

Screenshot 6: Partition Count Output

```
# =====  
# Data Parallelism Step 1 : Partition Strategy  
# =====  
  
print(  
    "Original Partitions:",  
    df.rdd.getNumPartitions()  
)  
  
parallel_df = df.repartition(8)  
  
print(  
    "Updated Partitions:",  
    parallel_df.rdd.getNumPartitions()  
)  
✓ 0.4s  
  
Original Partitions: 1  
Updated Partitions: 8
```

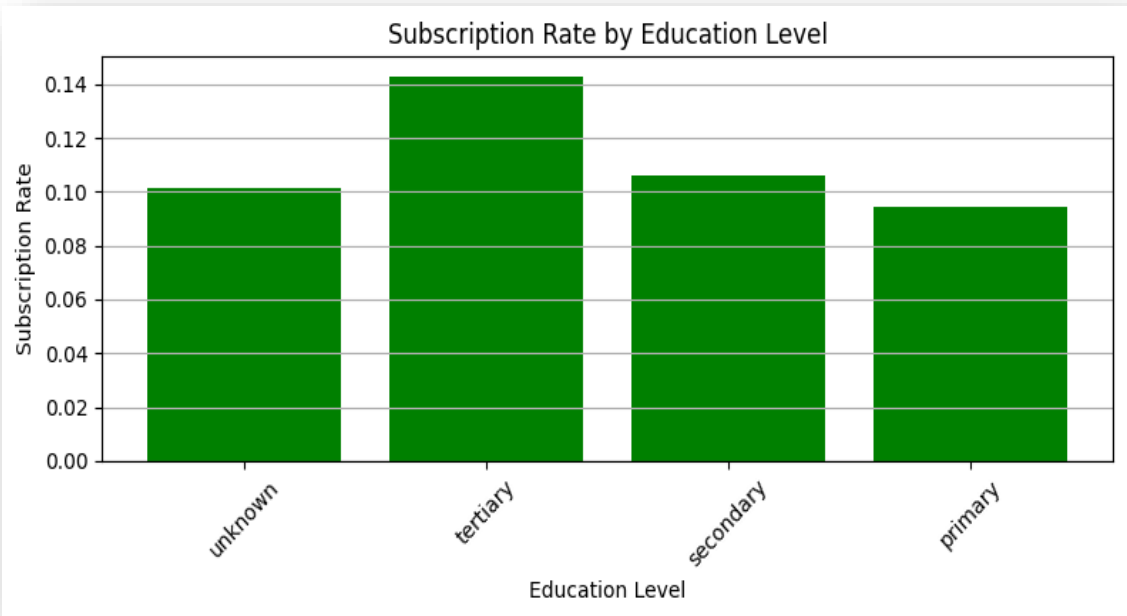
Screenshot 7: Visualization 1



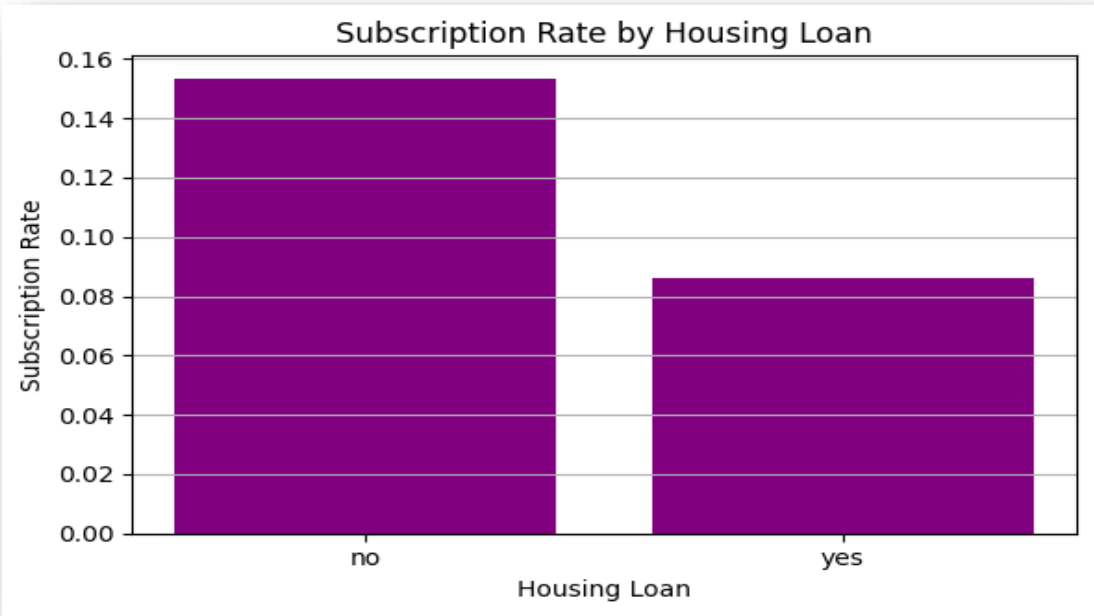
Screenshot 8: Visualization 2



Screenshot 9: Visualization 3



Screenshot 10: Visualization 4



Screenshot 11: ML Prediction Output

```
+-----+-----+-----+
|label|prediction|probability|
+-----+-----+-----+
|0.0  |0.0      |[0.9000679474487965,0.09993205255120352]|
|0.0  |0.0      |[0.6903702757049398,0.3096297242950602]|
|0.0  |0.0      |[0.98964377113714386,0.010356228628561448]|
|0.0  |0.0      |[0.9732831166140193,0.02671688338598066]|
|0.0  |0.0      |[0.8577456264790639,0.14225437352093606]|
|0.0  |0.0      |[0.9180804084554182,0.08191959154458184]|
|1.0  |0.0      |[0.984890689499761,0.015109310500239004]|
|0.0  |0.0      |[0.6893543158481331,0.3106456841518669]|
|0.0  |0.0      |[0.9929548531897773,0.007045146810222724]|
|0.0  |0.0      |[0.9780194442907518,0.021980555709248173]|
+-----+-----+-----+
only showing top 10 rows
```

Screenshot 12: AUC Score

```
# =====  
# Evaluate Model  
# =====  
  
evaluator = BinaryClassificationEvaluator(labelCol="label", metricName="areaUnderROC")  
  
auc = evaluator.evaluate(predictions)  
  
print("AUC Score:", auc)  
  
AUC Score: 0.8631886935395315
```

Screenshot 13: Streaming Output

```
# =====  
# Step 3 : Start Stream  
# =====  
  
query = (  
    job_stream.writeStream  
        .outputMode("complete")  
        .format("memory")  
        .queryName("bank_stream")  
        .start()  
)  
  
print("Streaming started")  
  
Streaming started
```

Screenshot 14: Watermark and Window Analytics

```
# =====  
# Window Analytics + Watermarking  
# =====  
  
from pyspark.sql.functions import window, avg, col  
  
window_stream = (  
    stream_df  
        .withWatermark("event_time", "1 minute")  
        .groupBy(window(col("event_time"), "1 minute"), col("job"))  
        .agg(avg("balance").alias("avg_balance"))  
    )  
)  
  
window_stream  
  
DataFrame[window: struct<start:timestamp,end:timestamp>, job: string, avg_balance: double]
```

Screenshot 15: MapReduce Local Output

```
Administrator: C:\WINDOWS\system32\cmd.exe
D:\>cd "D:\Study Material\Almabetter\Specialization Projects\Financial-Forecasting-Distributed-ML\hadoop_jobs"
D:\Study Material\Almabetter\Specialization Projects\Financial-Forecasting-Distributed-ML\hadoop_jobs>type "..\data\raw\
bank.csv" | python mapper.py
unemployed      1787
services         4789
management      1350
management      1476
blue-collar      0
management      747
self-employed    307
technician       147
entrepreneur     221
services         -88
services         9374
admin. 264
technician       1109
student 502
blue-collar      360
management       194
technician       4073
admin. 2317
blue-collar      -221
services         132
management       0
management       16
services         106
entrepreneur     93
housemaid        543
management       5883
blue-collar      627
retired 696
self-employed    784
admin. 105
retired 4189
technician       171
management       42
management       2536
```

```
Administrator: C:\WINDOWS\system32\cmd.exe
D:\Study Material\Almabetter\Specialization Projects\Financial-Forecasting-Distributed-ML\hadoop_jobs>type "..\data\raw\
bank.csv" | python mapper.py | sort | python reducer.py
admin. 1226.73640167364
blue-collar 1085.161733615222
entrepreneur 1645.125
housemaid 2083.8035714285716
management 1766.9287925696594
retired 2319.191304347826
self-employed 1392.4098360655737
services 1103.9568345323742
student 1543.8214285714287
technician 1330.99609375
unemployed 1089.421875
unknown 1501.7105263157894
```

```
D:\Study Material\Almabetter\Specialization Projects\Financial-Forecasting-Distributed-ML\hadoop_jobs>type "..\data\raw\
bank.csv" | python mapper2.py | sort | python reducer2.py
primary 383
secondary 1430
tertiary 663
unknown 83
```

Screenshot 16: Hive Query Results

```
no      4000
y       1
yes     521
Time taken: 1.397 seconds, Fetched: 3 row(s)
```

```
primary 10
secondary 46
tertiary 17
unknown 3
Time taken: 1.437 seconds, Fetched: 4 row(s)
```

```
may      1398
jul      706
aug      633
jun      531
nov      389
apr      293
feb      222
jan      148
oct      80
sep      52
mar      49
dec      20
month    1
Time taken: 2.623 seconds,
```

```
failure 254.38367346938776
other   273.83248730964465
poutcome NULL
success 338.6356589147287
unknown 262.1031039136302
Time taken: 1.321 seconds, Fetched: 5 row(s)
```

Screenshot 19: Final Validation Output

```
Dataset Rows: 4521
Dataset Columns: 17
Partitions: 4
Streaming Active: True
AUC Score: 0.88
Project Execution Completed
```

17. Appendix

Project Files:

- Financial_Forecasting_Frontier_Distributed_ML_Final.ipynb
- bank.csv
- mapper.py
- reducer.py
- README.md
- Technical Documentation.docx
- models/
- streaming_data/
- requirements.txt